

Деревья решений (Decision Trees) в scikit-learn

1. Основные концепции. Математическая формулировка

Дерево решений рекурсивно разбивает пространство признаков:

Узел m содержит данные Q_m с n_m выборками

Разбиение $\theta = (j, t_m)$ - признак j и порог t_m

Качество разбиения:

$$G(Q_m, \theta) = (n_{\text{left}}/n_m) \cdot H(Q_{\text{left}}) + (n_{\text{right}}/n_m) \cdot H(Q_{\text{right}})$$

2. Критерии качества. Для классификации

$$p_{mk} = (1/n_m) \cdot \sum I(y = k) \quad \# \text{Доля класса } k \text{ в узле } m$$

$$H(Q_m) = \sum p_{mk}(1 - p_{mk}) \quad \# \text{Джисси}$$

$$H(Q_m) = -\sum p_{mk} \log(p_{mk}) \quad \# \text{Энтропия (Log Loss)}$$

Для регрессии

MSE (среднее квадратичное отклонение)

$$y_m = (1/n_m) \sum y$$

$$H(Q_m) = (1/n_m) \sum (y - y_m)^2$$

$$H(Q_m) = (1/n_m) \sum |y - \text{median}(y)_m|$$

MAE (среднее абсолютное отклонение)

3. Практическая реализация. Задача 1: Классификация на Iris

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text

from sklearn.model_selection import train_test_split

from sklearn.metrics import classification_report, confusion_matrix

import matplotlib.pyplot as plt

# Загрузка данных

iris = load_iris()

X, y = iris.data, iris.target

# Разделение на обучающую и тестовую выборки

X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.3, random_state=42
)
# Создание и обучение модели
clf = DecisionTreeClassifier(
    max_depth=3,      # ограничиваем глубину
    min_samples_split=5, # мин. образцов для разбиения
    min_samples_leaf=2, # мин. образцов в листе
    criterion='gini',   # критерий Джини
    random_state=42
)
clf.fit(X_train, y_train)
# Предсказание
y_pred = clf.predict(X_test)
y_pred_proba = clf.predict_proba(X_test)
# Оценка
print("Точность:", clf.score(X_test, y_test))
print("\nОтчет по классификации:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
# Визуализация дерева
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.show()
# Текстовое представление
print(export_text(clf, feature_names=iris.feature_names))

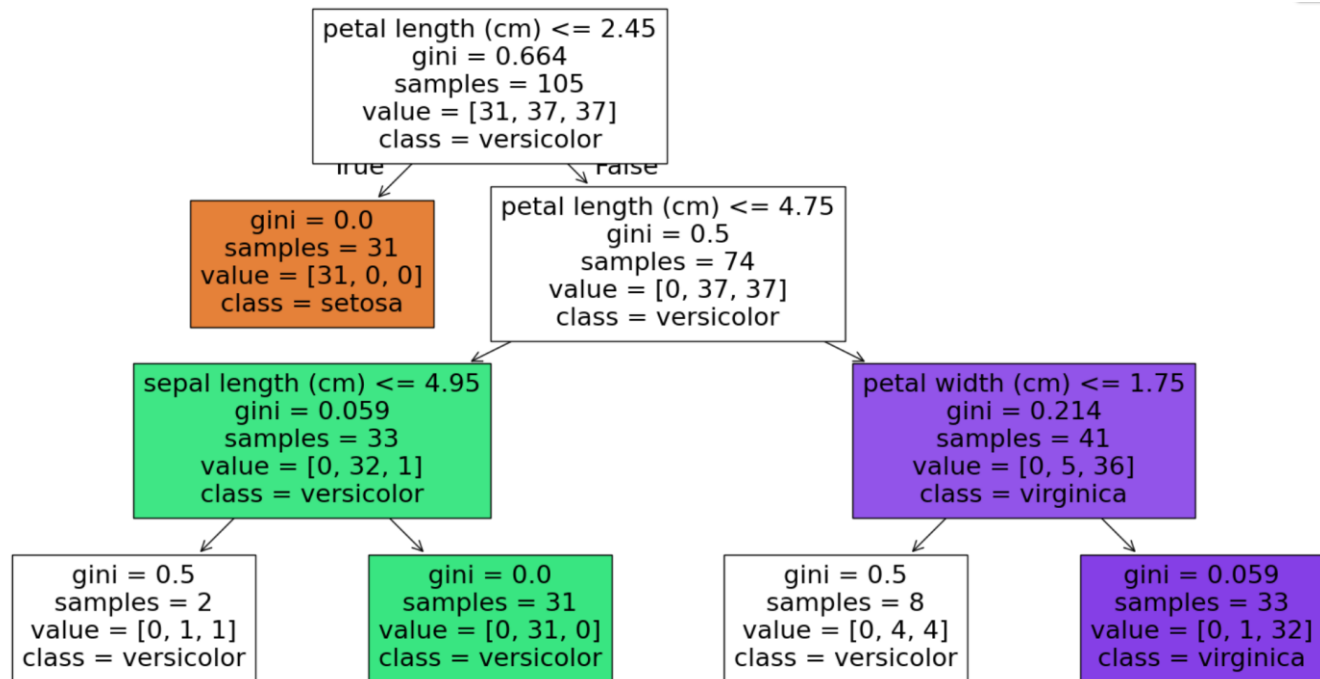
```

Точность: 1.0

Отчет по классификации:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19

versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45



```

|--- petal length (cm) <= 2.45
| |--- class: 0
|--- petal length (cm) > 2.45
| |--- petal length (cm) <= 4.75
| | |--- sepal length (cm) <= 4.95
| | | |--- class: 1
| | |--- sepal length (cm) > 4.95
| | | |--- class: 1
| |--- petal length (cm) > 4.75
| | |--- petal width (cm) <= 1.75
| | | |--- class: 1
| | |--- petal width (cm) > 1.75
| | | |--- class: 2
  
```

Задача 2: Регрессия (синусоида)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Генерация данных
np.random.seed(42)
X = np.sort(5 * np.random.rand(80, 1), axis=0)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])

# Разделение
X_train, X_test = X[:60], X[60:]
y_train, y_test = y[:60], y[60:]

# Сравнение деревьев разной глубины
depths = [2, 5, 10]
plt.figure(figsize=(15, 5))
for i, depth in enumerate(depths, 1):
    # Обучение
    reg = DecisionTreeRegressor(
        max_depth=depth,
        criterion='squared_error',
        min_samples_split=5,
        random_state=42
    )
    reg.fit(X_train, y_train)

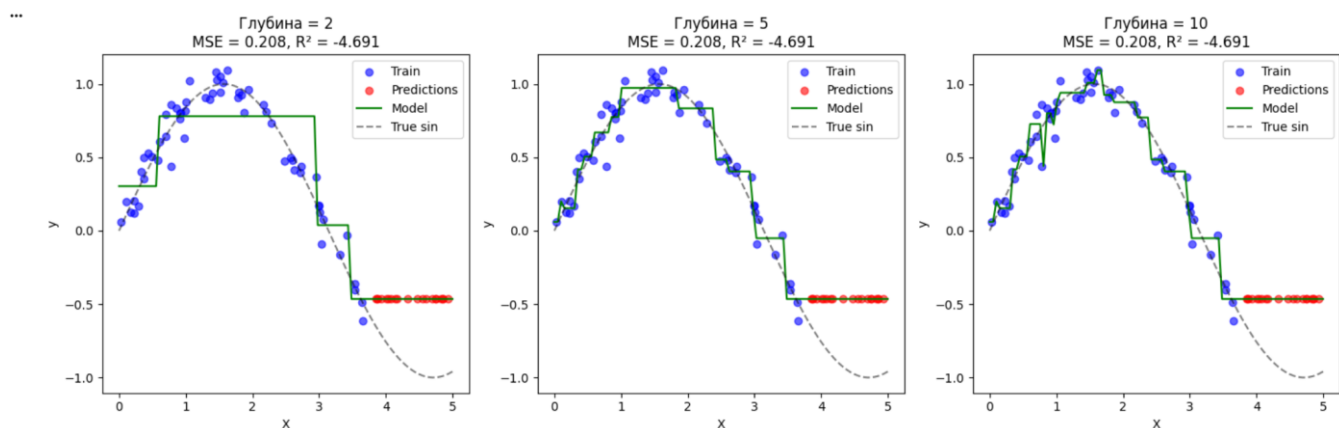
    # Предсказание
    y_pred = reg.predict(X_test)

    # Метрики
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
```

```

# Визуализация
plt.subplot(1, 3, i)
X_plot = np.linspace(0, 5, 100).reshape(-1, 1)
y_plot = reg.predict(X_plot)
plt.scatter(X_train, y_train, c='blue', label='Train', alpha=0.6)
plt.scatter(X_test, y_pred, c='red', label='Predictions', alpha=0.6)
plt.plot(X_plot, y_plot, 'g-', label='Model')
plt.plot(X_plot, np.sin(X_plot), 'k--', label='True sin', alpha=0.5)
plt.title(f'Глубина = {depth}\nMSE = {mse:.3f}, R² = {r2:.3f}')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.tight_layout()
plt.show()

```



Задача 3: Обрезка дерева (Cost-complexity pruning)

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Генерация данных
X, y = make_classification(
    n_samples=500, n_features=10, n_informative=5,

```

```

    n_redundant=0, random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# Обучение дерева без обрезки
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
print(f"Без обрезки - узлов: {clf.tree_.node_count}, "
      f"глубина: {clf.tree_.max_depth}")
# Получение пути обрезки
path = clf.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas
# Обучение деревьев с разными alpha
clfs = []
for ccp_alpha in ccp_alphas:
    clf = DecisionTreeClassifier(random_state=42, ccp_alpha=ccp_alpha)
    clf.fit(X_train, y_train)
    clfs.append(clf)
# Оценка качества
train_scores = [clf.score(X_train, y_train) for clf in clfs]
test_scores = [clf.score(X_test, y_test) for clf in clfs]
# Визуализация
fig, ax = plt.subplots(2, 1, figsize=(10, 8))
ax[0].plot(ccp_alphas, train_scores, 'b-', label='Train')
ax[0].plot(ccp_alphas, test_scores, 'r-', label='Test')
ax[0].set_xlabel('alpha')
ax[0].set_ylabel('Accuracy')
ax[0].legend()

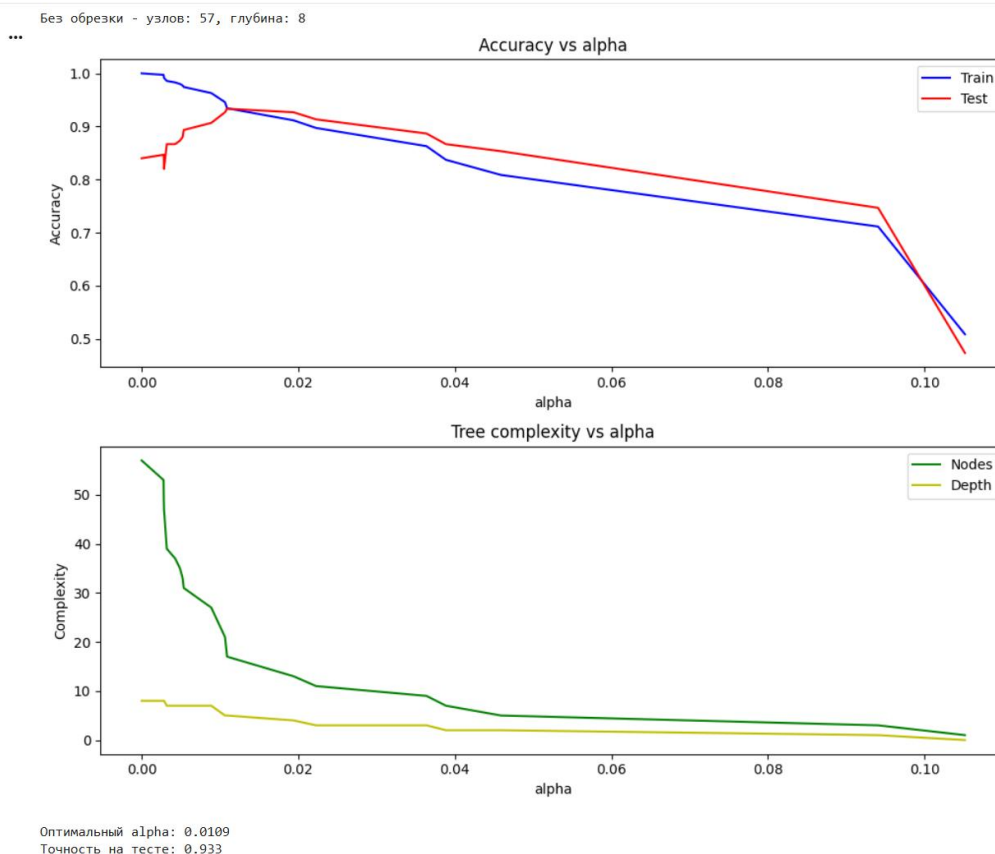
```

```

ax[0].set_title('Accuracy vs alpha')
node_counts = [clf.tree_.node_count for clf in clfs]
depth_counts = [clf.tree_.max_depth for clf in clfs]
ax[1].plot(ccp_alphas, node_counts, 'g-', label='Nodes')
ax[1].plot(ccp_alphas, depth_counts, 'y-', label='Depth')
ax[1].set_xlabel('alpha')
ax[1].set_ylabel('Complexity')
ax[1].legend()
ax[1].set_title('Tree complexity vs alpha')
plt.tight_layout()
plt.show()

# Выбор оптимального alpha
best_idx = np.argmax(test_scores)
best_alpha = ccp_alphas[best_idx]
print(f"\nОПТИМАЛЬНЫЙ alpha: {best_alpha:.4f}")
print(f"Точность на тесте: {test_scores[best_idx]:.3f}")

```



4. Ключевые параметры

<i>Параметр</i>	<i>Описание</i>	<i>Рекомендация</i>
<i>max_depth</i>	Макс. глубина дерева	Начать с 3-5
<i>min_samples_split</i>	Мин. образцов для разбиения	02.окт
<i>min_samples_leaf</i>	Мин. образцов в листе	01.май
<i>criterion</i>	Функция качества	'gini' или 'entropy'
<i>ccp_alpha</i>	Параметр обрезки	Подбирать через path

5. Интерпретация результатов. Важность признаков

Важность признаков

```
importances = clf.feature_importances_
```

```
for name, importance in zip(iris.feature_names, importances):
```

```
    print(f'{name}: {importance:.3f}')
```

Визуализация

```
plt.figure(figsize=(8, 4))
```

```
plt.bar(iris.feature_names, importances)
```

```
plt.title('Важность признаков')
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

Структура дерева # Информация о дереве

```
n_nodes = clf.tree_.node_count
```

```
children_left = clf.tree_.children_left
```

```
children_right = clf.tree_.children_right
```

```
feature = clf.tree_.feature
```

```
threshold = clf.tree_.threshold
```

```
print(f'Всего узлов: {n_nodes}')
```

```
for i in range(n_nodes):
```

```
    if children_left[i] != children_right[i]: # не лист
```

```
        print(f'Узел {i}: признак {feature[i]} <= {threshold[i]:.2f}')
```


6. Выводы

1. **Деревья решений** - интерпретируемые модели, склонные к переобучению
2. **Ключевые параметры** для контроля сложности: `max_depth`, `min_samples_leaf`
3. **Обрезка** (`ccp_alpha`) помогает найти баланс между смещением и дисперсией
4. **Важность признаков** дает понимание влияния переменных
5. **Визуализация** помогает понять логику принятия решений